Gauss Elimination Method

```
function x = gaussElimination(A,b)
n = length(b);
Aug = [A b];
% Forward elimination
for k = 1:n-1
   for i = k+1:n
      factor = Aug(i,k)/Aug(k,k);
      Aug(i,k:n+1) = Aug(i,k:n+1) - factor*Aug(k,k:n+1);
   end
end
% Back substitution
x = zeros(n,1);
x(n) = Aug(n,n+1)/Aug(n,n);
for i = n-1:-1:1
x(i) = (Aug(i,n+1) - Aug(i,i+1:n)*x(i+1:n))/Aug(i,i);
end
end
```

Gaussian Elimination With Partial Pivoting

```
function x = GEPartialPivoting(A,b)
n = length(b);
Aug = [A b];
for k = 1:n-1
  [~, m] = max(abs(Aug(k:n,k)));
   m = m + k - 1;
   Aug([k m],:) = Aug([m k],:);
   for i = k+1:n
      factor = Aug(i,k)/Aug(k,k);
      Aug(i,:) = Aug(i,:) - factor*Aug(k,:);
   end
end
x = zeros(n,1);
x(n) = Aug(n,n+1)/Aug(n,n);
for i = n-1:-1:1
   x(i) = (Aug(i,n+1) - Aug(i,i+1:n)*x(i+1:n)) / Aug(i,i);
end
end
```

Jacobi Method

```
function x = jacobi_method(A,b,tol,max_iter)
```

```
n = length(b);
x = zeros(n,1);
x_old = x;
for k = 1:max_iter
    for i = 1:n
        sum = 0;
        for j = 1:n
      if j ~= i
         sum = sum + A(i,j)*x_old(j);
      end
    end
        x(i) = (b(i) - sum)/A(i,i);
     end
    if norm(x-x_old,inf) < tol
    break;
  end
   x_old = x;
  end
end
```

Gauss–Seidel Method

```
function x = gauss_seidel(A,b,tol,max_iter)
n = length(b);
x = zeros(n,1);
for k = 1:max_iter
   x_old = x;
   for i = 1:n
      sum1 = A(i,1:i-1)*x(1:i-1);
      sum2 = A(i,i+1:n)*x_old(i+1:n);
      x(i) = (b(i) - sum1 - sum2)/A(i,i);
   end
   if norm(x-x_old,inf) < tol
      break;
   end
end
end
```

General Program

```
function x = solve_linear_system(A,b)

% Step 1: Check matrix size

[n,m] = size(A);

if n ~= m

   error('Matrix must be square');

end

% Step 2: Check determinant

if det(A) == 0

   error('System does not have a unique solution');

end

% Step 3: Solve using MATLAB built-in

x_builtin = A\b;

% Step 4: Solve using user-defined method

x_user = gaussian_elimination(A,b);

% Step 5: Display results

disp('Solution using user method:')

disp(x_user)

disp('Solution using MATLAB:')

disp(x_builtin)

% Step 6: Compute error

err = norm(x_user - x_builtin);

fprintf('Error between methods = %e\n',err);
```

```
x = x_user;

end
```

Example Problem

Solve:

$$\begin{aligned} x^2 + y^2 - 4 &= 0 \\ x - y - 1 &= 0 \end{aligned}$$

```
% Newton-Raphson for Nonlinear System

clc;

clear;

% Initial guess

X = [1; 1];   % [x; y]

% Tolerance and iterations

tol = 1e-6;

max_iter = 20;

for k = 1:max_iter

% Function vector

  F = [X(1)^2 + X(2)^2 - 4;

  X(1) - X(2) - 1];


 % Jacobian matrix

  J = [2*X(1), 2*X(2);

       1,     -1];
```

```
% Solve J * dX = -F
    dX = -J\F;

% Update
    X = X + dX;

% Check convergence
 if norm(dX) < tol
        fprintf('Converged in %d iterations\n', k);
        break;
    end
end
% Display result
fprintf('Solution:\n');
fprintf('x = %f\n', X(1));
fprintf('y = %f\n', X(2));
```